

“Universidad Peruana Los Andes”



Curso: Base De Datos 2

Tema: Resumen 01

Profesor:

Nombre y Apellido: Jeremy Joel Chancasanampa Garcia

Ciclo: 5

2026-1

Este resumen detallado integra la información de las fuentes sobre arquitecturas de bases de datos, modelos modernos como NoSQL y NewSQL, y las tendencias tecnológicas emergentes.

1. Arquitecturas Fundamentales de Bases de Datos

La estructura de las bases de datos se rige por modelos conceptuales que separan la lógica de los datos de su almacenamiento físico.

- **Modelo ANSI-SPARC:** Es un estándar conceptual que propone una división en **tres niveles**: el **nivel externo** (vistas personalizadas para usuarios), el **nivel conceptual** (estructura lógica global gestionada por el administrador o DBA) y el **nivel interno** (detalles del almacenamiento físico y hardware). El objetivo es proporcionar independencia de datos y ocultar la complejidad física a los usuarios finales.
- **Arquitecturas por Niveles (Tiers):** Dependiendo de la complejidad de la aplicación, las arquitecturas pueden ser de **un solo nivel** (todo en una máquina), **dos niveles** (cliente-servidor directo), **tres niveles** (con una capa intermedia o servidor de aplicaciones) o **n-niveles** (múltiples capas para lógica de negocio y acceso a datos).

2. Evolución hacia NoSQL, NewSQL y Bases de Datos Vectoriales

El crecimiento masivo de datos ha impulsado la transición desde los sistemas relacionales (RDBMS) tradicionales hacia modelos más escalables.

- **NoSQL (Not Only SQL):** Surgió para resolver limitaciones de los RDBMS en escalabilidad horizontal y el alto costo de operaciones como los *joins*. Se clasifican en almacenes de **Clave-Valor** (extremadamente rápidos), **Columnares** (ideales para analítica), **Orientadas a Documentos** (flexibles, suelen usar JSON) y **Orientadas a Grafos** (enfocadas en relaciones complejas).
- **NewSQL:** Estos sistemas buscan combinar el rendimiento y escalabilidad de NoSQL con las **garantías ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad) de las bases de datos relacionales tradicionales, siendo ideales para cargas de trabajo de procesamiento de transacciones en línea (OLTP).
- **Bases de Datos Vectoriales:** Diseñadas para almacenar y buscar datos no estructurados (imágenes, texto) convertidos en **vectores de alta dimensión**. Son fundamentales en la **IA generativa** y modelos de lenguaje (LLM), utilizando métricas como la **similitud coseno** para encontrar información semánticamente relacionada.

3. Sistemas Distribuidos y Microservicios

Las arquitecturas modernas priorizan la disponibilidad y la flexibilidad mediante la distribución de recursos.

- **Bases de Datos Distribuidas:** Funcionan como una única unidad lógica pero almacenan datos en múltiples nodos interconectados, lo que ahorra costes y

mejora la tolerancia a fallos. Pueden ser **homogéneas** (mismo hardware/software en todos los nodos) o **heterogéneas**.

- **Patrones de Microservicios:** La arquitectura de microservicios descompone aplicaciones monolíticas en servicios pequeños e independientes. Un patrón clave es la **Base de Datos por Microservicio**, que garantiza que cada servicio sea autónomo y pueda escalar de forma independiente, aunque introduce desafíos en la consistencia de los datos.
- **Serverless (Sin Servidor):** En este modelo, el proveedor de la nube gestiona automáticamente la infraestructura, ejecutando código (funciones) bajo demanda. Sus principales ventajas son la **escalabilidad automática** y el pago exclusivo por el tiempo de ejecución, aunque puede presentar latencia en el "arranque en frío".

4. Gestión de Concurrencia y Limitación de Tasa (Rate Limiting)

En entornos distribuidos, es crítico controlar el tráfico y evitar condiciones de carrera.

- **Algoritmos de Rate Limiting:** Se utilizan para proteger recursos contra picos de tráfico. El algoritmo de **Ventana Deslizante (Rolling Window)** es preferido por su precisión en el tiempo, implementándose frecuentemente con **Redis Sorted Sets** y **scripts de Lua** para asegurar que las operaciones de conteo sean atómicas y evitar inconsistencias en el acceso simultáneo.
- **Teorema CAP:** En sistemas distribuidos, se debe elegir entre Consistencia, Disponibilidad y Tolerancia al Particionamiento. Muchos sistemas de limitación de tasa eligen el modelo **AP (Disponibilidad y Tolerancia al Particionamiento)**, aceptando una consistencia eventual a cambio de que el servicio nunca deje de responder.

5. Tendencias para 2026

El panorama de los datos se dirige hacia una automatización e integración profunda.

- **Arquitecturas Modernas:** Se espera el dominio del modelo **Lakehouse** (flexibilidad de data lakes con rendimiento de data warehouses) y el enfoque **Data Mesh** (tratar los datos como un producto por dominios de negocio).
- **Automatización e IA:** La ingeniería de datos se automatizará mediante **DataOps**, y la inteligencia de decisiones (Decision Intelligence) permitirá que los sistemas recomienden o ejecuten acciones en tiempo real basándose en modelos de aprendizaje automático.
- **Gobernanza Adaptativa:** Se evolucionará hacia una gobernanza basada en código (**Governance as Code**), con políticas que se evalúan en tiempo real según el rol del usuario y la sensibilidad del dato.